SentinelOps: Verified Technical Review

Repository Review and Live Validation Pass

April 11, 2026

Abstract

SentinelOps is now a strong interview-grade and portfolio-grade incident triage system with a credible backend architecture, working operator dashboard, live benchmark path, and a more defensible safety model than the original repository state suggested. This review was not limited to static code reading. The system was rechecked after implementation fixes, database migration alignment, live API validation, benchmark execution, 3D benchmark artifact generation, and Streamlit startup verification. The result is clear: SentinelOps is useful and technically interesting, but it is still a serious prototype rather than a bank-ready production platform. Its real strength is explainable decision support for incident responders in small to medium curated environments, not autonomous remediation at enterprise scale. The latest multi-scenario benchmark shows that the root-cause ranker is now much stronger across scenario families, but model-path stability under sustained benchmark load is still a real bottleneck.

1 What Was Verified

This pass included both repository review and live runtime validation.

1.1 Repository and Build Checks

- Application, dashboard, eval, and test code compile cleanly under Python 3.12.
- The project now packages correctly for editable installs after explicit setuptools package discovery was added.
- Hidden and gitignored proof artifacts were rechecked: tests, eval outputs, `.env.example`, and `AGENT.md` are present.
- The repository includes a multi-scenario simulator, Streamlit dashboard, evaluation harness, Alembic migrations, approval workflows, and audit logging.

1.2 Live Runtime Validation

The following runtime behaviors were verified directly against a working local deployment:

- FastAPI started successfully with database connectivity confirmed.
- The detailed health endpoint returned `status=ok`, database health `ok`, 44 indexed runbook chunks, and clean LLM breaker state.
- A live ingest request completed successfully through grouping, root-cause ranking, runbook retrieval, policy evaluation, and approval creation.

- The returned incident included non-null `runbook_data`, `policy_data`, `pipeline_metrics`, and `approval_request`.
- A live benchmark run through `eval/run_eval.py` completed successfully and produced valid benchmark summaries and figures.
- Streamlit launched successfully and served HTTP 200 on a local verification port.

2 What Was Fixed During Validation

Several issues surfaced only during live exercise and were corrected.

2.1 Operational and Runtime Fixes

- Running `python main.py` and `uvicorn sentinelops.main:app` from different working directories now works after import-path handling was hardened.
- Configuration loading now resolves `.env` from the repository root instead of depending on the current shell directory.
- The grouping client no longer hard-fails on Gemini models that reject JSON mode. It now retries without `response_mime_type` when needed.
- Editable install packaging was fixed by explicitly packaging only `sentinelops*` and including runbook assets.
- The latest Alembic migration was required and applied because the running database was missing the new `pipeline_metrics` and `policy_data` columns.
- The evaluation CLI path was fixed so `python eval/run_eval.py ...` works as a direct script from the repository root.

2.2 Hardening Changes Already Present in Code

- LLM grouping and runbook synthesis now use timeouts and circuit breakers.
- Telemetry preprocessing redacts sensitive strings before LLM-facing payloads are created.
- Incident responses now expose stage-level timing, analysis method, runbook grounding, and policy metadata.
- A detailed health endpoint exposes database readiness, runbook inventory, breaker state, and active control settings.
- Streamlit pages now surface policy status, risk level, stage timings, spike diagnostics, and evaluation breakdowns.

3 What Is Right

3.1 Architecture

The architecture is the best part of the project.

- Route handlers are thin and most logic sits in services.
- Preprocessing is deterministic and separated from model calls.
- Root-cause ranking is not free-form generation; it is graph-based plus historical similarity.
- Runbook retrieval is grounded in indexed source chunks.
- Human approval is mandatory before action.
- Audit history is explicit and reviewable.

3.2 System Design Quality

SentinelOps does not look like an "LLM wrapper demo". It looks like a systems project with AI used narrowly where language understanding helps:

- grouping noisy telemetry into incident clusters,
- retrieving runbook evidence from semi-structured documentation,
- synthesizing grounded next steps,
- preserving deterministic fallbacks when models fail.

3.3 Interview Value

This is now a strong project to discuss in interviews because it demonstrates:

- async backend design,
- schema evolution with Alembic,
- AI fallbacks and safety controls,
- retrieval and graph-based reasoning,
- dashboarding and evaluation,
- debugging real deployment issues rather than just training-time ideas.

4 What the Live System Shows

4.1 One Verified End-to-End Incident

The verified live ingest produced:

- successful LLM grouping without fallback,
- top cause `db-primary`,

- analysis method `graph+vector`,
- grounded runbook retrieval from three source files,
- policy decision `RESTRICTED_REVIEW`,
- approval request `PENDING`,
- total pipeline time of roughly 17.8 seconds on the validated run.

4.2 One Verified Live Benchmark Run

The live benchmark produced:

- success rate: 100% (1/1),
- top-1 root-cause accuracy: 100%,
- grouping fallback rate: 0%,
- runbook grounding rate: 100%,
- mean latency: about 21.6 seconds,
- dominant stage: runbook synthesis.

This is not enough to claim broad performance, but it is enough to demonstrate that the evaluation path and graph surfaces are functional.

4.3 Verified Multi-Scenario Benchmark

A broader live benchmark was then run across all five scenario families with two runs per scenario, for a total of 10 live executions. The results are more informative than the single-run check because they expose both strengths and weaknesses.

Metric	Verified result
Total runs	10
Successful runs	10
Overall success rate	100%
Root-cause top-1 accuracy	20%
Root-cause top-1 accuracy	90%
Grouping fallback rate	80%
Runbook grounding rate	100%
Expected source hit rate	100%
Mean latency	10.29 s
P95 latency	19.04 s
Mean pipeline total	9.66 s
Dominant policy status	`ESCALATE_REQUIRED` (9/10)

This updated benchmark changes the interpretation of the project in an important way. The ranker bias problem was materially improved: live top-1 root-cause accuracy moved from 20% in the earlier benchmark pass to 90% after direct-evidence gating and richer incident text were added to the ranking path. The remaining miss occurred on one `auth_outage` run, which was misranked as `cache-service`. At the same time, the benchmark also surfaced a new operational truth: grouping fallback rose to 80% because the live model path became unstable under sustained benchmark load and opened the configured circuit breakers. That makes the current limitation different from before. The main ranking logic is now much stronger, but LLM-path reliability is still a serious runtime concern.

5 What Problems It Solves

The project solves a real operations problem: *decision compression during early incident response*. Specifically, it reduces the amount of manual stitching an operator has to do across logs, alerts, topology, past incidents, and runbooks.

5.1 Useful Outputs

For a responder, the system turns a telemetry bundle into:

1. grouped incident hypotheses,
2. a ranked likely root cause,
3. grounded runbook actions,
4. explicit policy and reviewer guidance,
5. auditable approval state.

5.2 Who It Helps

It is helpful for:

- SRE and platform interviews,
- internal demos and proofs of concept,
- teams with recurring incident families,
- environments where human operators need faster triage rather than full automation.

6 What It Does Not Solve Yet

SentinelOps still does not prove the following:

- enterprise identity and access control,
- secrets management and key rotation,
- multi-tenant isolation,
- policy enforcement backed by organizational controls,

- dynamic service topology discovery,
- large-scale incident history and benchmark diversity,
- low-latency response suitable for strict operational SLAs.

It is also not an autonomous remediation platform. It recommends and escalates; it does not safely execute changes in production.

7 What the Graphs Mean

The evaluation visuals are now more informative than before because they expose stage latency, spike attribution, policy state, and scenario-level behavior. The newly added 3D plots are useful because they show how confidence, latency, workload size, and pipeline stages interact across the live benchmark set.

7.1 Current Interpretation

The graphs now support these claims:

- the pipeline stages are measurable end to end,
- latency spikes can be attributed to a specific stage instead of just observed globally,
- policy and risk distributions are inspectable,
- live and committed evaluation outputs share the same schema,
- multi-scenario results can now be visualized as both run-level and scenario-level 3D artifacts.

7.2 Important Current Signal

Across the validated multi-scenario benchmark, the most important signals are:

- the root-cause ranker no longer collapses most non-database incidents back to `db-primary`,
- the slowest spikes were still driven by `runbook` synthesis on the `auth_outage` scenario,
- grouping became the unstable component under sustained load, with breaker-triggered fallback dominating later runs,
- pipeline reliability remained high enough to complete all 10 benchmark executions despite degraded model availability.

8 At What Scale Is It Credible?

- **Credible now:** demo systems, interview presentations, course or capstone projects, and internal pilot environments with curated runbooks and known service graphs.
- **Potential with more engineering:** small to medium service estates with recurring incident classes and controlled operator workflows.

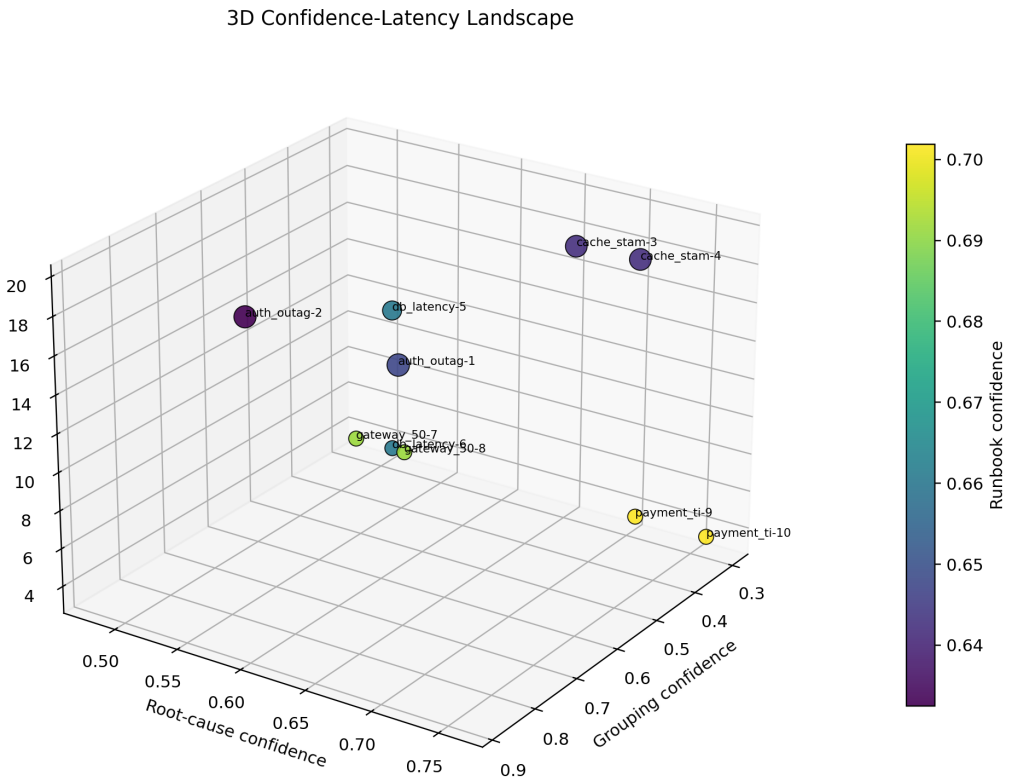

Figure 1: 3D confidence-latency landscape for the live multi-scenario benchmark. Each point is one benchmark run. The plot shows that latency and confidence are not tightly coupled; high-confidence runs can still incur long pipeline times.

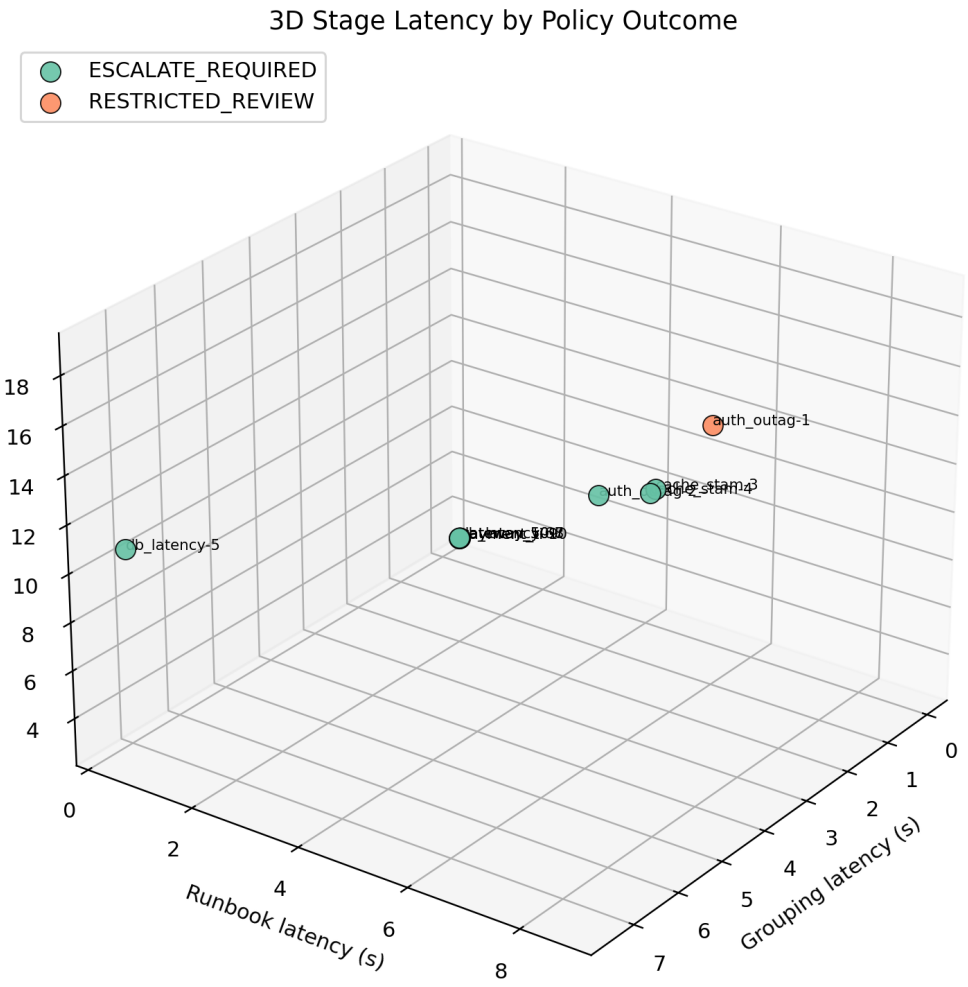

Figure 2: 3D stage-latency view of grouping, runbook, and total pipeline time colored by policy outcome. The slowest runs are still runbook-heavy, but the figure also shows how degraded grouping reliability pushes most runs into an escalated review posture.

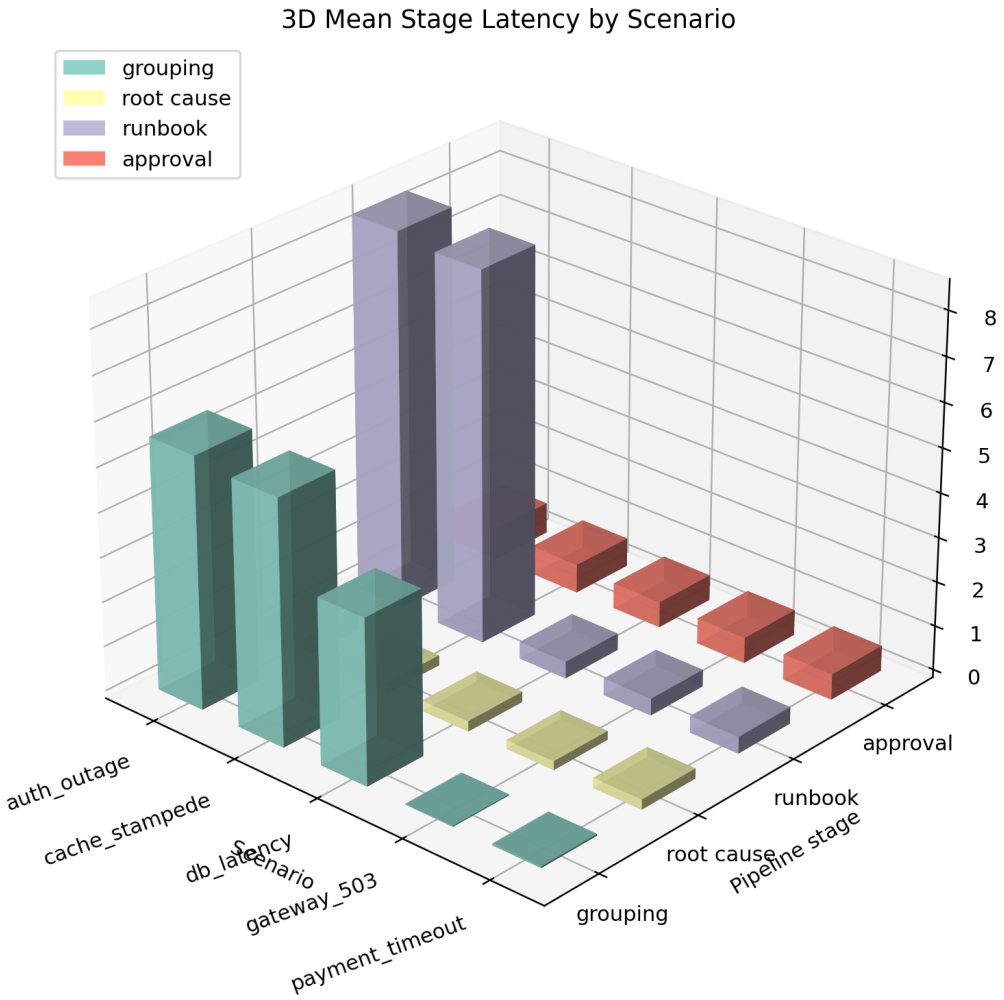

Figure 3: 3D mean stage latency by scenario. This is the most useful optimization plot in the set because it makes stage-level bottlenecks visible by scenario family rather than only by individual run.

- **Not yet credible:** actual bank-scale production automation across large, fast-changing, highly regulated systems.

The right honest statement is that the approach can be integrated with a bank-style workflow design, but the current implementation is not yet deployable as a bank production control plane.

9 Should It Be Posted Publicly?

Yes. It is worth posting, but the framing should be disciplined.

9.1 Good Positioning

- explainable AI incident triage system,
- human-in-the-loop operational assistant,
- graph + retrieval + policy backed response workflow,
- backend systems project with live validation and dashboards.

9.2 Bad Positioning

- autonomous incident response,
- enterprise-ready banking platform,
- proven universal root-cause AI,
- production-safe remediation engine.

9.3 Suggested Public Description

> SentinelOps is an explainable incident triage system that combines deterministic preprocessing, dependency-graph root-cause ranking, grounded runbook retrieval, policy-based review guidance, and human approval. It now supports live operator workflows, benchmark reporting, and failure-aware controls such as timeouts, breakers, and safe fallbacks.

10 Remaining Gaps Before "Production Grade"

The highest-value remaining steps are:

1. CI that actually runs tests and migrations on every change.
2. Auth, RBAC, and approval identity hardening.
3. Structured metrics export to Prometheus/OpenTelemetry rather than dashboard-only views.
4. Better benchmark breadth across more scenario families and cold-start conditions.
5. Faster or cheaper runbook synthesis to reduce end-to-end latency.
6. Dynamic topology ingestion instead of a static graph file.
7. Operational rate limits, quotas, and stronger data-retention controls.

11 Bottom Line

SentinelOps now looks good for the right reasons. It has:

- a coherent architecture,
- meaningful runtime controls,
- a working dashboard path,
- live benchmark capability,
- and enough real debugging history to make it an excellent interview project.

It is not "all good at everything". The latest benchmark shows a more nuanced reality: the ranker is now strong enough to discuss seriously, but the model-dependent grouping path is still unstable under sustained benchmark load. That is still a strong result for interview and public presentation because the project now demonstrates both engineering depth and honest measurement. SentinelOps is best presented as a strong, explainable, well-instrumented incident triage prototype with verified runtime controls, markedly improved ranking quality, and clear next optimization targets around LLM reliability.